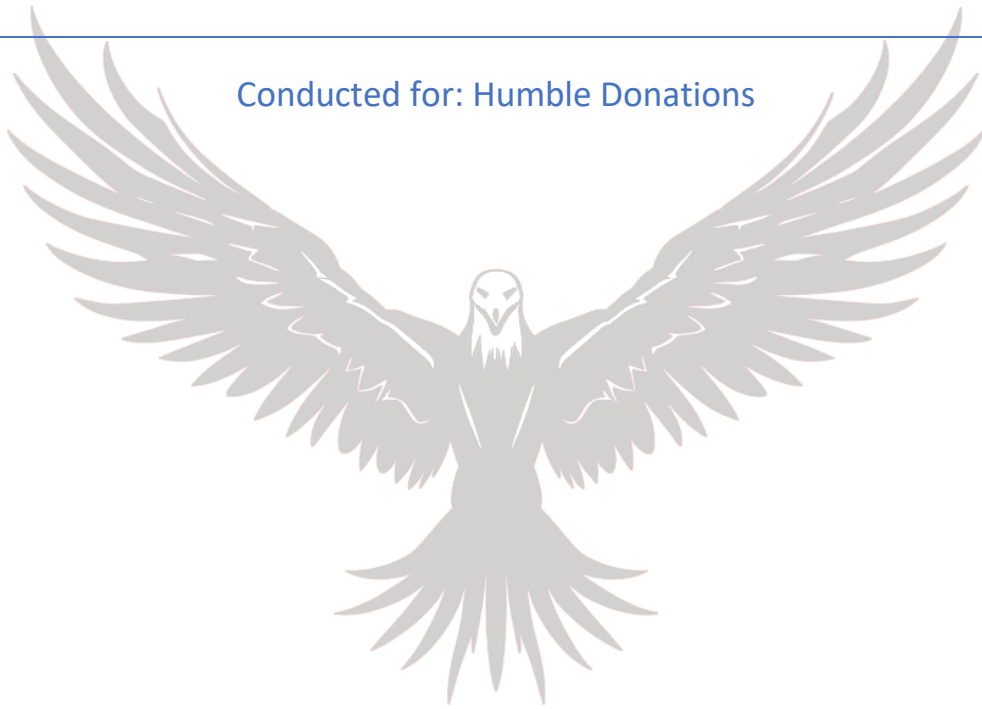




SMART CONTRACT AUDIT

Conducted for: Humble Donations



VULSIGHT

Contents

Executive Summary:..... 3

Project Background: 3

Audit Scope:..... 3

Contract Summary: 3

Severity Definitions: 4

Technical Checks: 4

Code Quality and Documentation: 5

Audit Findings:..... 5

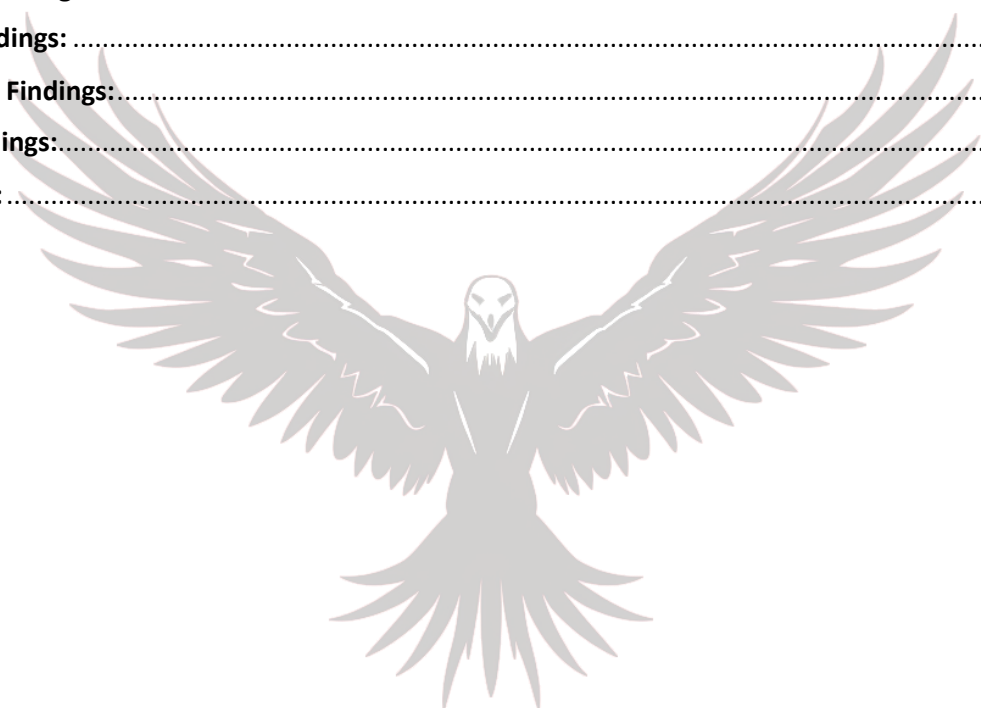
 Critical findings:..... 5

 High Findings: 5

 Medium Findings:..... 5

 Low findings:..... 5

Disclaimer: 6



VULSIGHT

Executive Summary:

The client contacted our Consulting Firm to conduct a smart contract audit on their Solidity based smart contract. The HumbleDonationsToken smart contract is a simple ERC20 token smart contract that is designed to facilitate charitable donations on the Ethereum blockchain.

The smart contract was found to be relatively simple. Our comprehensive audit revealed no vulnerabilities.

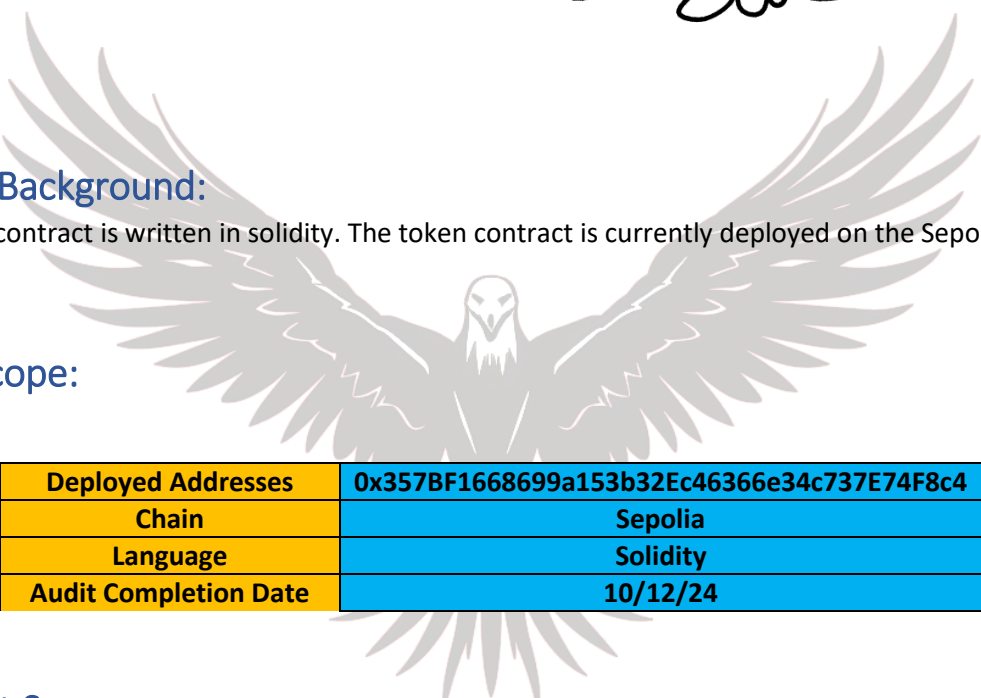
Audit conducted by: Waleed Ahmed



Project Background:

The smart contract is written in solidity. The token contract is currently deployed on the Sepolia testnet.

Audit Scope:



Deployed Addresses	0x357BF1668699a153b32Ec46366e34c737E74F8c4
Chain	Sepolia
Language	Solidity
Audit Completion Date	10/12/24

Contract Summary:

- **Contract type:** The HumbleDonationsToken is an ERC20 token contract that implements multiple OpenZeppelin extensions for enhanced functionality and security.
- **Token standard implementation:** The contract implements the ERC20 standard with additional features through OpenZeppelin's extensions including ERC20Burnable, ERC20Pausable, ERC20Permit, and ERC20Votes.
- **Token specifications:** The token has the symbol "HDT" (Humble Donations Token) with a maximum supply of 100 million tokens, which are initially minted to the contract owner.
- **Access control:** The contract utilizes OpenZeppelin's Ownable pattern to restrict administrative functions (minting, pausing) to the contract owner only.
- **Pausability:** The contract includes pause/unpause functionality that can temporarily halt all token transfers, providing an emergency stop mechanism.

- **Advanced features:** The contract implements ERC20Permit for gasless approvals and ERC20Votes for governance functionality, showing consideration for modern token utility cases.

We found:

- 0 Critical risk finding
- 0 High risk findings
- 0 Medium risk findings
- 0 Low risk findings

Severity Definitions:

Risk Level	Description
Critical	Any kind of vulnerability that could lead to direct token or monetary loss
High	Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.
Medium	Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss
Low	Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Matching Project Specifications	Passed
Logical Vulnerabilities	Passed
Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included one smart contract, which was written in Solidity programming language. The code is well indented and clear in nature.

Audit Findings:

Critical findings:

Zero critical findings were found in the smart contract.

High Findings:

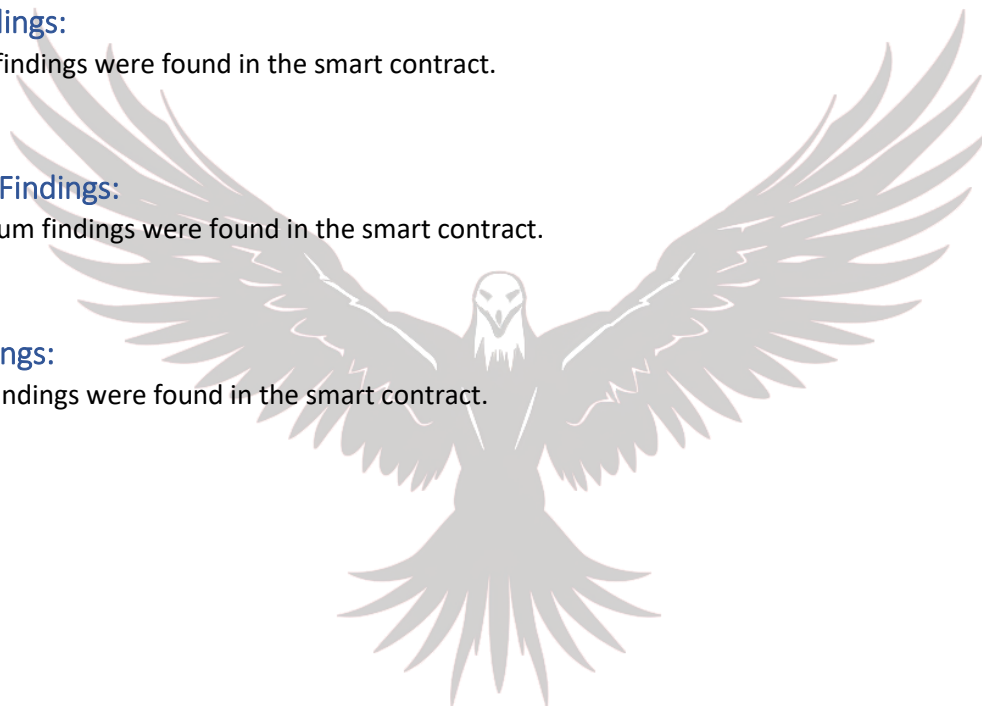
Zero high findings were found in the smart contract.

Medium Findings:

Zero medium findings were found in the smart contract.

Low findings:

Zero low findings were found in the smart contract.



VULSIGHT

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. While we have done our best in testing this smart contract, it is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.



VULSIGHT